

Teoria Współbieżności

Laboratorium 6

Problem czytelników i pisarzy

Kacper Bieniasz

13listopada 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadania 1

Problem czytelników i pisarzy proszę rozwiązać przy pomocy: semaforów i zmiennych warunkowych. Proszę wykonać pomiary dla różnej ilości czytelników (10-100) i pisarzy (od 1 do 10). W sprawozdaniu proszę narysować 3D wykres czasu w zależności od liczby wątków i go zinterpretować.

3 Rozwiązanie zadania 1

Kod (1) zawiera implementację rozwiązania problemu pisarzy i czytelników. Rozważany problem jest bardzo powszechnie rozważany. Polega na dostępie do zasobu w celu odczytania, pod warunkiem, że nie jest on wtedy modyfikowany. Czytelnicy mogą mieć dostęp jednocześnie jednak, aby pisarz miał dostęp musi to mieć zasób na wyłączność. Bardzo ważne jest, aby nie zagłodzić zarówno czytelników jak i pisarzy. Dlatego trzeba zagwarantować sprawiedliwy dostęp do zasobów dla obu grup.

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.concurrent.Semaphore;
6
7 public class ReadersWriters {
8     private static final Semaphore writeLock = new Semaphore(1);
9     private static final Semaphore mutex = new Semaphore(1);
10    private static int readCount = 0;
11    static class Reader extends Thread {
12        private long waitingTime;
13        @Override
14        public void run() {
15            long startWait = System.currentTimeMillis();
16            try {
17                mutex.acquire();
18                waitingTime = System.currentTimeMillis() - startWait;
19                readCount++;
20                if (readCount == 1) {
```

```

21         writeLock.acquire();
22     }
23     mutex.release();
24     Thread.sleep(50);
25     mutex.acquire();
26     readCount--;
27     if (readCount == 0) {
28         writeLock.release();
29     }
30     mutex.release();
31 } catch (InterruptedException e) {
32     Thread.currentThread().interrupt();
33 }
34 }
35 public long getWaitingTime() {
36     return waitingTime;
37 }
38 }
39 static class Writer extends Thread {
40     private long waitingTime;
41     @Override
42     public void run() {
43         long startWait = System.currentTimeMillis();
44         try {
45             writeLock.acquire();
46             waitingTime = System.currentTimeMillis() - startWait;
47             Thread.sleep(100);
48             writeLock.release();
49         } catch (InterruptedException e) {
50             Thread.currentThread().interrupt();
51         }
52     }
53     public long getWaitingTime() {
54         return waitingTime;
55     }
56 }
57 public static void main(String[] args) throws InterruptedException, IOException {
58     int[] readersArray = new int[100];
59     for (int i = 10; i < readersArray.length+10; i++) {
60         readersArray[i-10] = i;
61     }
62     int[] writersArray = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
63     try (FileWriter writer = new FileWriter("readers_writers_results.csv")) {
64         writer.write("Readers, Writers, AverageWaitingTime(ms)\n");
65         for (int readers : readersArray) {
66             for (int writers : writersArray) {
67                 System.out.println("Readers: " + readers + ", Writers: " + writers);
68                 List<Thread> threads = new ArrayList<>();
69                 List<Reader> readersList = new ArrayList<>();
70                 List<Writer> writersList = new ArrayList<>();
71                 for (int i = 0; i < readers; i++) {
72                     Reader reader = new Reader();
73                     threads.add(reader);
74                     readersList.add(reader);
75                 }
76                 for (int i = 0; i < writers; i++) {
77                     Writer writerThread = new Writer();
78                     threads.add(writerThread);
79                     writersList.add(writerThread);
80                 }
81                 for (Thread thread : threads) {
82                     thread.start();

```

```

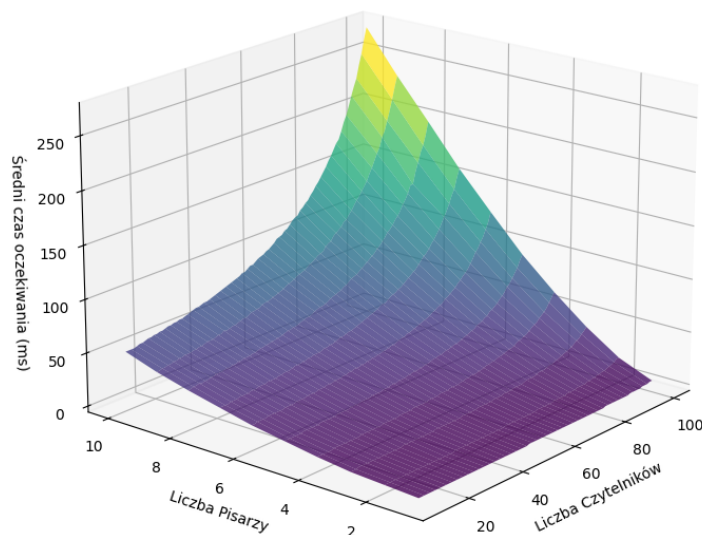
83     }
84     for (Thread thread : threads) {
85         thread.join();
86     }
87     long totalWaitingTime = 0;
88     for (Reader reader : readersList) {
89         totalWaitingTime += reader.getWaitingTime();
90     }
91     for (Writer writerThread : writersList) {
92         totalWaitingTime += writerThread.getWaitingTime();
93     }
94     long averageWaitingTime = totalWaitingTime / (readers + writers);
95     writer.write(readers + "," + writers + "," + averageWaitingTime + "\n");
96     System.out.println("Average waiting time: " + averageWaitingTime + " ms\n");
97 }
98 }
99 }
100 System.out.println("Results saved to readers_writers_results.csv");
101 }
102 }

```

Kod Java 1: Implementacja rozwiązania problemu czytelników i pisarzy

Klasa **ReadersWriters** realizują całość zadania. Metoda *main* odpowiada za parametry wykonania programu, wybiera liczbę czytelników i pisarzy tworzy odpowiednie wątki oraz zapisuje statystyki wykonania programu do pliku w celu wykonania wykresu. Klasy statyczne **Reader** oraz **Writer** mierzą czas jaki czekają w celu wyznaczenia średniego czasu oczekiwania w zależności od liczby wątków. Każda z nich realizuje swoje zadanie odpowiednio odczytu i zapisu. Wykres (1) przedstawia średni czas oczekiwania na dostęp do zasobu przez wątek w zależności od liczby pisarzy i czytelników.

Średni czas oczekiwania w Zależności od Liczby Pisarzy i Czytelników



Rysunek 1: Wykres średniego czasu oczekiwania na zasób przez wątek

4 Wnioski z zadania 1

Wykonanie programu przeprowadziłem dla liczby pisarzy $\{1, \dots, 10\}$ oraz dla liczby czytelników $\{10, \dots, 100\}$. Wykres dostarcza dodatkowych informacji na temat omawianego problemu. Najważniejszym parametrem który wpływa na wzrost czasu jest liczba pisarzy. Oni nie mogą pracować równocześnie tak jak ma to miejsce w przypadku czytelników. Wykres jest dość gładki co również pokazuje, że większy wpływ na wydajność ma liczba pisarzy.

5 Treść zadania 2

1. Proszę zaimplementować listę, w której każdy węzeł składa się z wartości typu Object, referencji do następnego węzła oraz zamka (lock).
2. Proszę zastosować metodę drobnoziarnistego blokowania do następujących metod listy:
 - boolean contains(Object o); //czy lista zawiera element o
 - boolean remove(Object o); //usuwa pierwsze wystąpienie elementu o
 - boolean add(Object o); //dodaje element o na końcu listy
3. Proszę porównać wydajność tego rozwiązania w stosunku do listy z jednym zamkiem blokującym dostęp do całości. Należy założyć, że koszt czasowy operacji na elemencie listy (porównanie, wstawianie obiektu) może być duży - proszę wykonać pomiary dla różnych wartości tego kosztu.

6 Rozwiązanie zadania 2

W celu realizacji tego zadania wykorzystam technikę zwaną blokowaniem drobnoziarnistym. Jest to sposób synchronizacji części struktury. Opis blokowania drobnoziarnistego wygląda następująco:

- Zamek (lock) jest przydatny wtedy, gdy operacje zamykania/otwierania nie mogą być umieszczone w jednej metodzie lub bloku synchronized. Przykładem jest zakładanie blokady (lock) na elementy struktury danych, np. listy. Podczas przeglądania listy stosujemy następujący algorytm:
 1. zamknij zamek na pierwszym elemencie listy
 2. zamknij zamek na drugim elemencie
 3. otwórz zamek na pierwszym elemencie
 4. zamknij zamek na trzecim elemencie
 5. otwórz zamek na drugim elemencie
 6. powtarzaj dla kolejnych elementów
- Dzięki temu unikamy konieczności blokowania całej listy i wiele wątków może równocześnie przeglądać i modyfikować różne jej fragmenty.

Kod (2) zawiera implementację takiej struktury danych, która realizuje polecenie zadania 2.

```
1 import java.util.concurrent.locks.Lock;
2 import java.util.concurrent.locks.ReentrantLock;
3 public class FineGrainedList {
4     private class Node {
5         Object value;
6         Node next;
7         Lock lock;
8         Node(Object value) {
9             this.value = value;
10            this.next = null;
11            this.lock = new ReentrantLock();
12        }
13    }
14    private final Node head;
```

```

15 public FineGrainedList() {
16     head = new Node(null);
17 }
18 public boolean contains(Object o) {
19     Node current = head;
20     current.lock.lock();
21     try {
22         Node next = current.next;
23         while (next != null) {
24             next.lock.lock();
25             try {
26                 if (next.value.equals(o)) {
27                     return true;
28                 }
29                 current.lock.unlock();
30                 current = next;
31                 next = next.next;
32             } finally {
33                 current.lock.unlock();
34             }
35         }
36         return false;
37     } finally {
38         current.lock.unlock();
39     }
40 }
41 public boolean remove(Object o) {
42     Node current = head;
43     current.lock.lock();
44     try {
45         Node next = current.next;
46         while (next != null) {
47             next.lock.lock();
48             try {
49                 if (next.value.equals(o)) {
50                     current.next = next.next;
51                     return true;
52                 }
53                 current.lock.unlock();
54                 current = next;
55                 next = next.next;
56             } finally {
57                 next.lock.unlock();
58             }
59         }
60         return false;
61     } finally {
62         current.lock.unlock();
63     }
64 }
65 public boolean add(Object o) {
66     Node current = head;
67     current.lock.lock();
68     try {
69         while (current.next != null) {
70             current = current.next;
71             current.lock.lock();
72             current.lock.unlock();
73         }
74         Node newNode = new Node(o);
75         current.next = newNode;
76         return true;

```

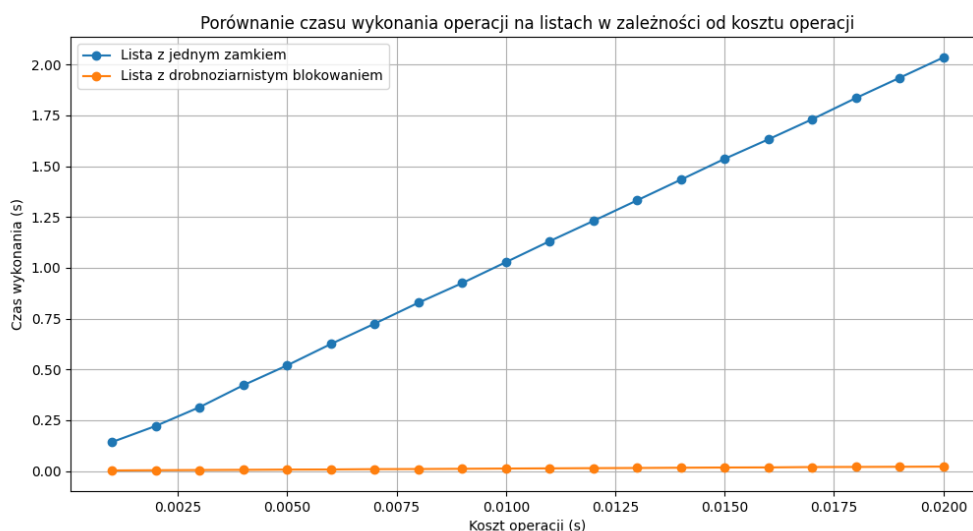
```

77     } finally {
78         current.lock.unlock();
79     }
80 }
81 }

```

Kod Java 2: Implementacja struktury danych z zadania 2

Dzięki zastosowaniu mechanizmu blokowania ziarnistego, możemy wykonywać opisane w poleceniu, możemy wykonywać wiele operacji na strukturze niemal równolegle. W celu przeprowadzenia wydajności musiałem sztucznie wydłużyć czas operacji, realizuję to poprzez dodatkowe wykorzystanie metody *Tread.sleep(aktualny_koszt)* dla każdej operacji. Operacje na liście prowadziło 10 wątków, każdy z nich realizował po 100 w kolejności *add(val)*, *contains(val)* oraz *remove(val)* Dla każdego kosztu zapisuję czas wykonania wszystkich operacji. W celu porównania z strukturą nie implementującą blokowania drobnoziarnistego, zmodyfikowałem kod (2) tak, aby blokował dostęp do całej struktury. Wykres (2) przedstawia czas wykonania w zależności od kosztu dla obu implementacji list.



Rysunek 2: Wykres czasów wykonania dla różnych wartości kosztu

7 Wnioski z zadania 2

Zastosowanie blokowania drobnoziarnistego, ma ogromny wpływ na wydajności operacji. Mając dużą strukturę na której wykonujemy wiele operacji taki mechanizm znacząco poprawiłby wydajność. Wykres (2) pokazuje, że omawiany mechanizm zachowuje wzorcową wydajność. Krzywa odpowiadająca strukturze **FineGrainedList** jest niemalże płaska, z prawie niezauważalnym wzrostem. W przypadku prymitywnej listy mechanizmem blokującym całą strukturę, krzywa przypomina funkcję liniową. Prosty mechanizm jakim jest blokowanie drobnoziarniste daje nam bardzo dobre wyniki, dzięki czemu jest dobrym narzędziem do poprawy wydajności. Jednak ma on pewne założenia, które rozważana struktura danych go implementująca musi posiadać.

Literatura

- [1] <https://home.agh.edu.pl/~funika/tw/lab6/RW-problem.pdf>
- [2] https://en.wikipedia.org/wiki/Readers%E2%80%93writers_problem
- [3] <https://mieszkomakuch.github.io/problemy-synchronizacyjne/doc/readers-writers.html>